



TRABAJO DE FINAL DE GRADO

GRADO EN INTELIGENCIA ROBÓTICA

Estudio e implementación del control de péndulo invertido sobre carro en un sistema de tiempo real

Estudiante:
Miguel Porcar Vicent

Tutor:
Ignacio Peñarrocha Alós

Fecha de presentación: Junio de 2026
Curso académico: 2025/2026

Agradecimientos:

Palabras clave:

CASTELLÓN, a XX de XXXXXXX de 2025

Todos los derechos reservados

ÍNDICE GENERAL

1	Introducción	9
1.1	Motivaciones	9
1.2	Objetivos	9
1.3	Planificación	9
1.4	Estimación de costes	9
2	Análisis	11
2.1	Requisitos funcionales	12
2.2	Requisitos no funcionales	13
2.3	Diseño del sistema	13
2.3.1	Modelado cinemático y dinámico	14
2.3.2	Estrategia de control Swing-up	14
2.3.3	Estabilización por realimentación del estado	14
2.3.4	Lógica de conmutación	14
2.4	Arquitectura del sistema	15
2.4.1	Arquitectura de la simulación	15
2.4.2	Arquitectura del sistema real	16
2.4.3	Especificación de componentes de hardware	16
3	Desarrollo	17
3.1	Simulación	17
3.2	Sistema físico	17
4	Resultados	19
4.1	Resultados en simulación	19

4.2	Resultados en sistema físico	19
5	Conclusiones y posible trabajo futuro	21
	Bibliografía	23
6	Apéndices	25

ÍNDICE DE FIGURAS

2.1	Arquitectura de control	13
2.2	Arquitectura de la simulación	15
2.3	Arquitectura del sistema real	16

ÍNDICE DE TABLAS

1. Introducción

1.1 Motivaciones

En caso de que haya motivo(s) concreto(s) para la realización del proyecto.

1.2 Objetivos

Objetivos que se espera alcanzar al inicio del proyecto.

1.3 Planificación

Lista de tareas, relación entre ellas y duración estimada Posibles desviaciones de la planificación Diagrama de Gantt

1.4 Estimación de costes

Estimación del coste económico que supondría la adquisición del material necesario (hardware y software) para la realización del proyecto, los salarios involucrados y otros aspectos como alquileres de espacios y energía.

2. Análisis

En este capítulo se realiza un estudio detallado de los pilares que sustentan el desarrollo del sistema de control para el péndulo invertido. El análisis se articula en tres ejes fundamentales:

- **Análisis de requisitos:** Donde se formalizan las capacidades operativas (funcionales) y las restricciones técnicas, de seguridad y rendimiento (no funcionales) que debe satisfacer el sistema.
- **Diseño del sistema:** Se describe la estrategia lógica y matemática adoptada, detallando el modelo dinámico, las leyes de control híbridas y la lógica de conmutación necesaria para el *swing-up* y la estabilización.
- **Arquitectura del sistema:** Se define la infraestructura tanto de simulación como de implementación real, especificando el flujo de datos, la integración con ROS y la selección de los componentes de hardware que garantizan la viabilidad del proyecto.

2.1 Requisitos funcionales

Los requisitos funcionales describen las acciones, cálculos y manipulaciones de datos específicos que el sistema debe realizar para cumplir su propósito. Para el control del péndulo invertido sobre carro se definen mediante sus entradas, comportamiento y salidas:

■ RF1. Modelado y simulación de la dinámica:

- *Entradas:* Parámetros físicos del sistema (masas, inercias, coeficientes de fricción) y señal de control simulada.
- *Comportamiento:* Resolución matemática de las Ecuaciones Diferenciales Ordinarias (EDO) que rigen la dinámica no lineal del mecanismo en el entorno MATLAB/Simulink.
- *Salidas:* Estado cinemático simulado del sistema (posiciones y velocidades articulares).

■ RF2. Visualización y telemetría en ROS:

- *Entradas:* Estado cinemático del sistema (proveniente de la simulación o del hardware real) y archivo de descripción geométrica URDF.
- *Comportamiento:* Traducción y publicación de los datos mediante el puente de comunicación de ROS Toolbox para actualizar el árbol de transformadas (TF) del robot.
- *Salidas:* Visualización 3D actualizada en tiempo real mediante la interfaz RViz (tópico `/joint_states`).

■ RF3. Control de inyección de energía (Swing-up):

- *Entradas:* Posición y velocidad angular actual del péndulo ($q, \dot{q}$).
- *Comportamiento:* Cálculo y ejecución de una ley de control basada en energía para balancear el péndulo desde su posición estable ($q = \pi$ rad) hasta el punto de linealización del sistema ($q = 0$ rad).
- *Salidas:* Acción de control (u).

■ RF4. Control de estabilización (Realimentación del estado):

- *Entradas:* Vector de estado del sistema ($x = [p, \dot{p}, q, \dot{q}]^T$).
- *Comportamiento:* Estabilización del sistema sobre el punto de linealización utilizando control por realimentación del estado.
- *Salidas:* Acción de control (u).

■ RF5. Adquisición y decodificación de señales:

- *Entradas:* Señales eléctricas de pulsos en cuadratura (A y B) provenientes de los encoders del motor y del péndulo.
- *Comportamiento:* Procesamiento y conteo de los pulsos digitales mediante el hardware específico.
- *Salidas:* Valores digitales discretos de posición y velocidad interpretables por la ley de control.

2.2 Requisitos no funcionales

Los requisitos no funcionales imponen condiciones y restricciones a la implementación del diseño:

- **RNF1. Restricciones de rendimiento (Determinismo y tiempo real):** El bucle de control debe ejecutarse dentro de una rutina de interrupción periódica en un microcontrolador, garantizando un periodo de muestreo estable y un *jitter* despreciable para la viabilidad del control discreto.
- **RNF2. Restricciones de fiabilidad (Robustez):** El sistema debe rechazar perturbaciones externas moderadas y ser tolerante a las discrepancias producidas por dinámicas no modeladas (fricciones no lineales y holguras mecánicas).
- **RNF3. Restricciones de seguridad (Gestión de fallos):**
 - *Seguridad mecánica:* El software debe implementar una parada de emergencia si las lecturas de posición indican que el carro excede los límites físicos del riel. En caso de fallo en el software de parada se debe disponer de finales de carrera para cortar la alimentación del actuador.
 - *Seguridad eléctrica:* Se debe garantizar el aislamiento galvánico entre la etapa de control (3.3V) y la etapa de potencia (24V).
- **RNF4. Condiciones de implementación (Abstracción HAL):** El código desarrollado debe hacer uso de la capa de abstracción de hardware (HAL) ofrecida por el fabricante del microcontrolador, asegurando la portabilidad, estandarización y eficiencia del firmware.

2.3 Diseño del sistema

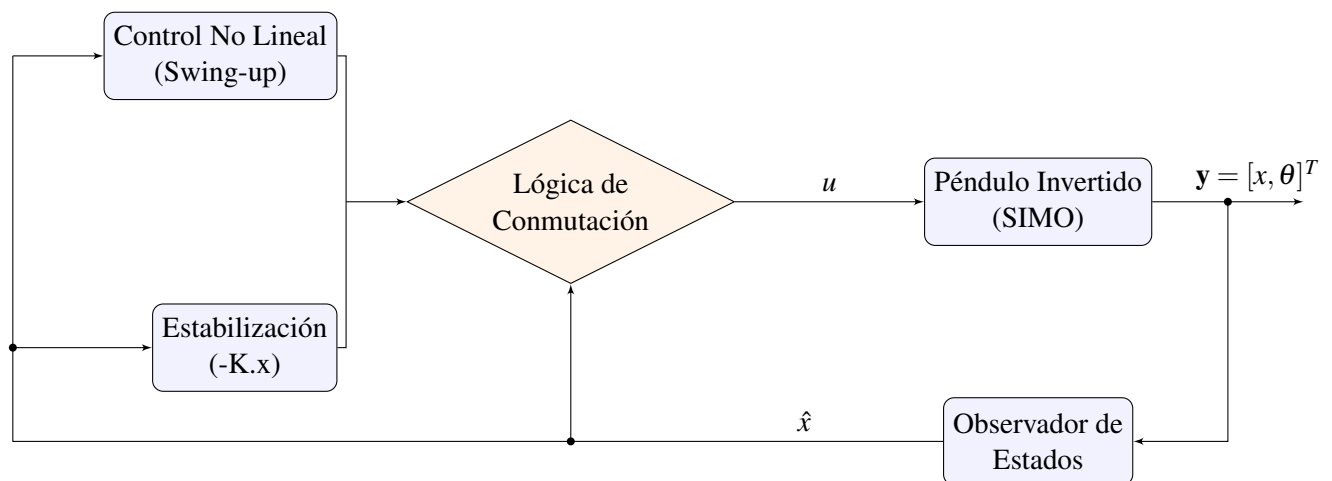


Figura 2.1: Arquitectura de control

La arquitectura de control propuesta en la Figura 2.1 se aplica sobre un sistema subactuado tipo SIMO (Single Input Multiple Output). Para la etapa de realimentación, se implementa un observador del estado por inversión. Mediante este enfoque, los estados del sistema se obtienen exclusivamente a partir de las mediciones de salida, dado que, en el caso concreto de sistemas robóticos, el estado coincide con las salidas cinemáticas ($q, \dot{q}, \ddot{q}$).

2.3.1 Modelado cinemático y dinámico

El objetivo de esta sección es compactar las ecuaciones de movimiento del péndulo invertido sobre carro en la expresión Euler - Lagrange. Buscar un libro que haga el desarrollo.

2.3.2 Estrategia de control Swing-up

Aquí tenemos que explicar en que consiste el control de energía (podemos poner el símil del columpio que es muy visual.)

2.3.3 Estabilización por realimentación del estado

Aquí interesa linealizar las ecuaciones de Euler - Lagrange y definir los estados del sistema.

2.3.4 Lógica de conmutación

Finalmente, la lógica de conmutación atiende a un umbral angular. Este bloque de decisión monitoriza de forma continua la posición angular estimada del péndulo ($\hat{\theta}$). Si el péndulo ingresa dentro de una vecindad acotada en torno a la vertical absoluta (*el punto de linealización del sistema*), la lógica conmuta al control por realimentación del estado. En caso contrario, el sistema mantiene activo el controlador de swing-up para elevar el mecanismo. Este comportamiento híbrido se formaliza en el siguiente pseudocódigo:

Algoritmo 1 Lógica de Conmutación

Require: Vector de estados estimado $\hat{x} = [\hat{x}_c, \dot{\hat{x}}_c, \hat{\theta}, \dot{\hat{\theta}}]^T$, umbral angular θ_{th}

Ensure: Señal de control u a aplicar a la planta

```

1: loop
2:    $\hat{\theta} \leftarrow \hat{x}(3)$  ▷ Obtener posición angular actual
3:   if  $|\hat{\theta}| \leq \theta_{th}$  then
4:      $u \leftarrow -K\hat{x}$  ▷ Control de posición
5:   else
6:      $u \leftarrow f_{swing}(\hat{x})$  ▷ Control de energía
7:   end if
8:
9: end loop

```

2.4 Arquitectura del sistema

Para definir la arquitectura del sistema se ha hecho una división entre la parte de simulación y la implementación en un sistema real.

La arquitectura del sistema de simulación permite obtener una vista general del flujo de trabajo para el modelado y la validación antes de la implementación real. Por otro lado, la arquitectura del sistema real se centra en los periféricos involucrados en el control así como el proceso de discretización que se debe aplicar para controlar un sistema real utilizando un microcontrolador.

2.4.1 Arquitectura de la simulación

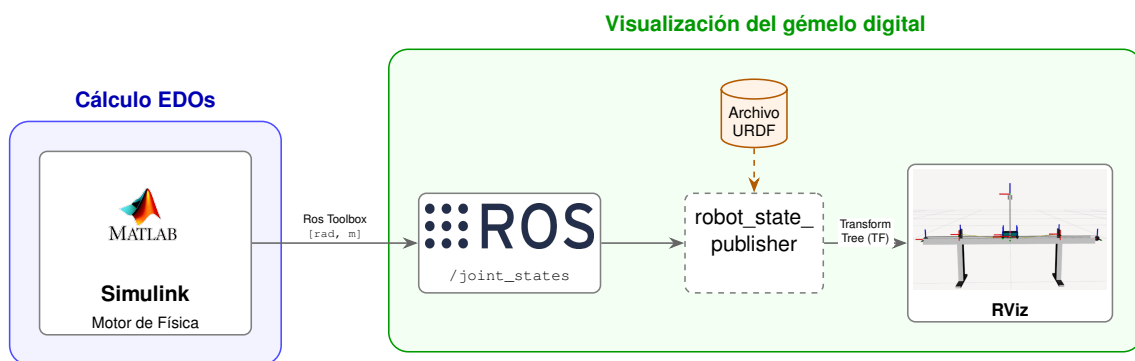


Figura 2.2: Arquitectura de la simulación

La Figura 2.2 detalla la arquitectura de software diseñada para la simulación del sistema. A continuación se desglosa cada elemento del esquema para conocer su función y como se conecta con el resto de la arquitectura:

- **Cálculo de EDOs (Motor de Física):** Implementado en el entorno MATLAB/Simulink, este bloque se encarga de resolver las Ecuaciones Diferenciales Ordinarias (EDO) que describen la dinámica del sistema. Simulink actúa como el motor de física, calculando en cada paso de tiempo los estados del robot (posiciones en radianes y metros). Estos datos se transmiten al ecosistema ROS mediante ROS Toolbox, publicando la información en el tópico */joint_states*.
- **Visualización, gemelo digital:** Este bloque recibe la información cinemática para representar el comportamiento del sistema en tiempo real. Está totalmente integrado en el entorno de ROS y se compone de varias piezas:
 - **Archivo URDF (Unified Robot Description Format):** Es un archivo XML que describe como es el robot o mecanismo que se quiere simular. Se especifican las visuales, las restricciones cinemáticas y los marcos de referencia de las articulaciones.
 - **Robot State Publisher:** Es el nodo de ROS que permite representar el movimiento del robot. Se suscribe al tópico */joint_states* y actualiza el árbol de transformadas de nuestro archivo URDF.
 - **RViz:** La última pieza de la arquitectura software, es la interfaz que permite visualizar de forma gráfica e intuitiva todo el flujo de datos como un modelo 3D en movimiento.

2.4.2 Arquitectura del sistema real

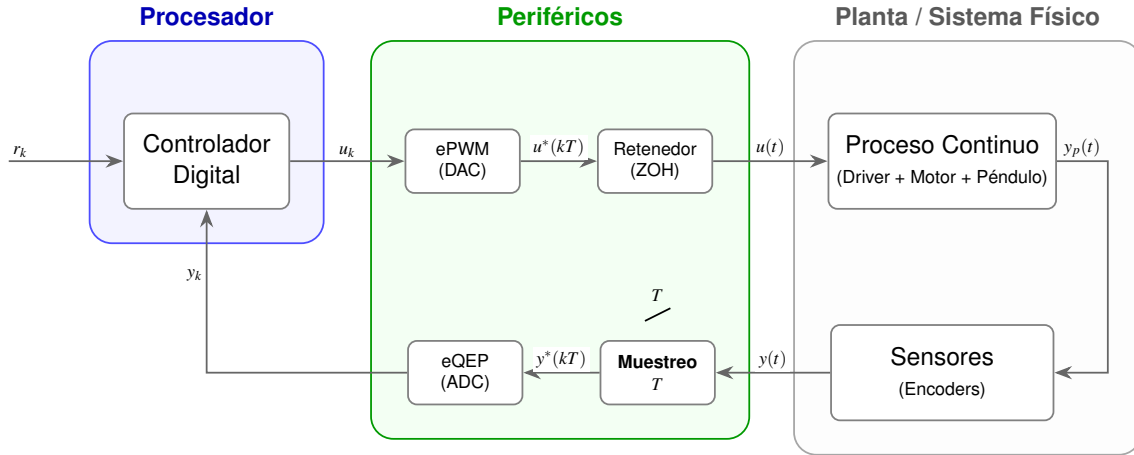


Figura 2.3: Arquitectura del sistema real

La Figura 2.3 presenta de forma esquemática la implementación del bucle de control en un microcontrolador.

Esta estructura permite observar la interacción entre el entorno digital (discreto) y el proceso físico (continuo). El núcleo del sistema es el **Controlador Digital**. Este procesador se encarga de ejecutar el algoritmo de control que procesa la señal de error entre la referencia (r_k) y la realimentación (y_k).

El concepto fundamental es el **periodo de muestreo (T)**. Este parámetro determina la periodicidad de la rutina de control, permitiendo sincronizar las etapas de medida, cálculo y actuación sobre la planta de manera precisa. Durante el desarrollo del proyecto se determinará el periodo de muestreo atendiendo a factores como la dinámica del sistema en bucle cerrado o la frecuencia de la señal PWM.

Finalmente la interacción entre el mundo digital y el mundo físico es posible mediante el uso de **periféricos** que permiten medir el estado del sistema o modificarlo. Diferenciamos en dos clases de periféricos:

- **Actuadores:** La señal de control digital (u_k) se convierte en una señal analógica o cuasi-analógica mediante el módulo *ePWM* (DAC, convertidor digital-analógico). Posteriormente, un bloque *Retenedor de orden cero* (ZOH) mantiene la señal constante durante el intervalo de muestreo, transformándola en la señal continua $u(t)$ que excita al Proceso Continuo (compuesto por el driver, el motor y el péndulo).
- **Sensores:** La respuesta del sistema ($y_p(t)$) es captada por los sensores (en este caso, encoders). La señal $y(t)$ es procesada por un bloque de muestreo, gobernado por el periodo T , y enviada al periférico *eQEP*. Este último actúa como un equivalente a un ADC (convertidor analógico-digital), transformando los pulsos del encoder en una señal digital y_k interpretable por el controlador.

2.4.3 Especificación de componentes de hardware

Aquí poner el microcontrolador que tenemos con sus características, el puente H y los encoders.

3. Desarrollo

Explicación de las actividades realizadas durante el desarrollo del proyecto. Se puede exponer en orden cronológico o por grupos de tareas. Hay que indicar los hitos alcanzados.

En esta sección solamente hay que aportar la información técnica estrictamente necesaria para comprender el trabajo realizado. Esta información debe, además, estar presentada de forma que sea inteligible por cualquiera no familiarizado con los aspectos técnicos del trabajo realizado. La información técnica puede ubicarse en los apéndices para ser consultada si se desea.

3.1 Simulación

1) Uso de parámetros proporcionados por el fabricante (rozamiento, inercias, etc.) para modelado del sistema en Matlab/Simulink.

2) Conexión exitosa entre ROS y Matlab mediante la herramienta ROS Toolbox, hay que comentar como se tubo que cambiar el DDS al cyclone para funcionar, la lógica de publicador subscriber, el diseño de un archivo URDF para la visualización de la unidad mecánica desde RViz. (Posibles vías de estudio como la migración de la simulación completamente a ROS)

3) Explicar también como mediante simulación se pudo obtener las ganancias necesarias para poder hacer control swin-up con PFL. Muy importante poner los diagramas de bloques de nuestro simulink (El de publicación de tópicos, el de dinámica del sistema y el de conmutación de control (Swing-Up to LQR))

4) Finalmente explicar como se acabo implementando una simulación discreta del sistema (uso de bucle for y mida de paso con el método de Euler o1). Sirve como puente a la implementación de algoritmos de control en microcontroladores.

3.2 Sistema físico

1) Selección de microcontrolador, Pte H y encoders. Explicar un poco la tecnología detrás de cada componente (Ejemplo: Sabemos que los encoders son HEDS-9140-A00, también sabemos que el PTE H tiene aislamiento galvánico mediante optoacopladores).

2)Firmware, comentar como se programa una DSP de texas instruments mediante el uso del IDE CCS y la capa de HAL C2000-ware. Utilizar el datashet del fabricante para justificar la configuración de ciertos registros. Módulos Epwm y Eqp muy interesante ver como liberan a la CPU de estas tareas.

3)Explicar como se programo la DSP con timers para ejecutar el bucel de control en una rutina de interrupción periódica (implica determinismo, nada del loop de Arduino)

4)(Lo que hay que acabar de hacer este més) Razonar experimentos de identificación de parámetros del sistema. La justificación es que los parámetros del fabricante no sabemos si son del todo ciertos al ser una máquina de hace 25 años. Algunos experimentos: Conversión de voltios a fuerza horizontal (encontrar la constante que relaciona ambas magnitudes), Identificación del rozamiento horizontal del carro, identificación del rozamiento y la inercia del péndulo.

5)Simular de nuevo con los parámetros obtenidos mediante identificación.

4. Resultados

- 4.1 Resultados en simulación
- 4.2 Resultados en sistema físico

5. Conclusiones y posible trabajo futuro

Trabajo a futuro → Implementación en HAL, uso de algoritmos no basados en modelo (Reinforcement Learning)

Bibliografía

6. Apéndices

Colocar aquí cualquier tipo de información susceptible de ser consultada para la correcta comprensión del documento y el proyecto que describe.